

```
#LAB
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
np.random.seed(42)
n = 200
locations = ['urban', 'suburban', 'rural']
df = pd.DataFrame({
    'area_sqft': np.random.randint(500, 4000, n),
    'num_rooms': np.random.randint(1, 7, n),
    'location': np.random.choice(locations, n),
    'age_years': np.random.randint(0, 40, n),
})
loc_map = {'urban': 1.5, 'suburban': 1.0, 'rural': 0.7}
noise = np.random.normal(0, 20000, n)
df['price'] = (
    df['area_sqft'] * 150 +
    df['num_rooms'] * 25000 -
    df['age_years'] * 2000 +
    df['location'].map(loc_map) * 50000 + noise).astype(int)
print(df.head(8).to_string(index=False))
print(f"\nDataset shape : {df.shape}")
print(df.describe()[['area_sqft', 'num_rooms', 'age_years', 'price']].round(0))
```

```
area_sqft  num_rooms  location  age_years  price
3674         4      rural        20  646067
1360         4      urban        35  300479
1794         6      urban         9  501609
1630         6      rural        36  355668
1595         3      urban         8  391991
3592         2      rural        23  566413
2138         4  suburban        34  410974
2669         1      urban        34  459191
```

```
Dataset shape : (200, 5)
   area_sqft  num_rooms  age_years  price
count    200.0     200.0     200.0   200.0
mean     2334.0         3.0        20.0  451548.0
std       992.0         2.0        11.0  154871.0
min       521.0         1.0         0.0   86373.0
25%      1519.0         2.0        10.0  326188.0
50%      2372.0         3.0        20.0  458282.0
75%      3230.0         5.0        30.0  575910.0
max      3999.0         6.0        39.0  781261.0
```

```
le = LabelEncoder()
df['location_enc'] = le.fit_transform(df['location'])
features = ['area_sqft', 'num_rooms', 'age_years', 'location_enc']
X = df[features]
y = df['price']
print(df[['location', 'location_enc']].drop_duplicates().sort_values('location_enc'))
print(f"\nFeature matrix shape : {X.shape}")
print(f"Target vector shape : {y.shape}")
```

```
   location  location_enc
0      rural            0
6  suburban            1
1      urban            2

Feature matrix shape : (200, 4)
Target vector shape : (200,)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print(f"Training samples : {len(X_train)} ({len(X_train)/len(X)*100:.0f}%)")
print(f"Testing samples : {len(X_test)} ({len(X_test)/len(X)*100:.0f}%)")
print(f"Feature names : {features}")
```

```
Training samples : 160 (80%)
Testing samples : 40 (20%)
Feature names : ['area_sqft', 'num_rooms', 'age_years', 'location_enc']
```

```
model = LinearRegression()
model.fit(X_train, y_train)
print("Learned Coefficients:")
print("-" * 45)
for feat, coef in zip(features, model.coef_):
```

```

direction = "↑" if coef > 0 else "↓"
print(f" {feat:15s}: Rs {coef:>10,.0f} {direction}")
print(f" {'intercept':15s}: Rs {model.intercept_:>10,.0f}")
print("-" * 45)

```

Learned Coefficients:

```

-----
area_sqft      : Rs      151 ↑
num_rooms      : Rs    23,169 ↑
age_years      : Rs    -2,262 ↓
location_enc    : Rs    18,114 ↑
intercept      : Rs    46,789
-----

```

```

y_pred = model.predict(X_test)
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print("=" * 40)
print(" MODEL EVALUATION REPORT")
print("=" * 40)
print(f"R² Score : {r2:.4f} {'Excellent' if r2>0.9 else ' Good' if r2>0.8 else 'Needs work'}")
print(f"MAE : Rs {mae:>10,.0f}")
print(f"RMSE : Rs {rmse:>10,.0f}")
print("=" * 40)
print("\nSample Predictions:")
print(f"{'Actual':>10} | {'Predicted':>10} | {'Error':>10}")
print("-" * 36)
for a, p in zip(y_test[:8], y_pred[:8]):
    print(f"{a:>10}, | {p:>10,.0f} | {p-a:>10,.0f}")

```

```

=====
MODEL EVALUATION REPORT
=====
R² Score : 0.9872 Excellent
MAE : Rs    13,439
RMSE : Rs    16,959
=====

```

Sample Predictions:

Actual	Predicted	Error
489,899	478,320	-11,579
601,329	572,877	-28,452
623,788	650,489	+26,701
291,458	271,086	-20,372
390,040	344,431	-45,609
123,884	134,545	+10,661
484,442	443,295	-41,147
562,823	556,350	-6,473

```

new_houses = pd.DataFrame({
    'area_sqft': [1300, 3500, 600, 1500],
    'num_rooms': [2, 5, 6, 7 ],
    'age_years': [7, 30, 28, 1 ],
    'location_enc': [3, 5, 4, 7 ],
    'location_lbl': ['Urban', 'Suburban', 'Rural', 'Urban'],
})
predictions = model.predict(new_houses[features])
print("\n HOUSE PRICE PREDICTIONS")
print("=" * 60)
for i, (_, row) in enumerate(new_houses.iterrows()):
    print(f" House {i+1}: {row.area_sqft} sqft | "
          f"{row.num_rooms} rooms | "
          f"{row.age_years}yr old | "
          f"{row.location_lbl}")
    print(f" Estimated Price → Rs {predictions[i]:>10,.0f}")
print()

```

HOUSE PRICE PREDICTIONS

```

=====
House 1: 1300 sqft | 2 rooms | 7yr old | Urban
Estimated Price → Rs    327,393

```

```

House 2: 3500 sqft | 5 rooms | 30yr old | Suburban
Estimated Price → Rs    712,385

```

```

House 3: 600 sqft | 6 rooms | 28yr old | Rural
Estimated Price → Rs    285,268

```

```

House 4: 1500 sqft | 7 rooms | 1yr old | Urban
Estimated Price → Rs    559,383

```

